IFPI
Floriano

Dominando as Funções em Java

A arte de não repetir código (Introdução a Métodos)

Prof. Ronaldo Borges

NotebookLM

O Caos da Repetição vs. A Elegância da Organização

```java
// Bloco para nota1
if (nota1 >= 7.0) {
    System.out.println("Aluno 1: Aprovado com nota " + nota1);
} else {
    System.out.println("Aluno 1: Reprovado com nota " + nota1);
}

// Bloco para nota2
if (nota2 >= 7.0) {
    System.out.println("Aluno 2: Aprovado com nota " + nota2);
} else {
    System.out.println("Aluno 2: Reprovado com nota " + nota2);
}

// Bloco para nota3
if (nota3 >= 7.0) {
    System.out.println("Aluno 3: Aprovado com nota " + nota3);
} else {
    System.out.println("Aluno 3: Reprovado com nota " + nota3);
}
// ... e assim por diante, repetindo a mesma lógica.
```

Código **propenso a erros**, exaustivo de ler e difícil de manter.

```java
verificarNota(nota1);
verificarNota(nota2);
verificarNota(nota3);
```

A lógica agrupada em um bloco reutilizável. O código se torna legível como um texto em português.

NotebookLM

A Metáfora: Pense na Função como uma Fábrica

Parâmetros

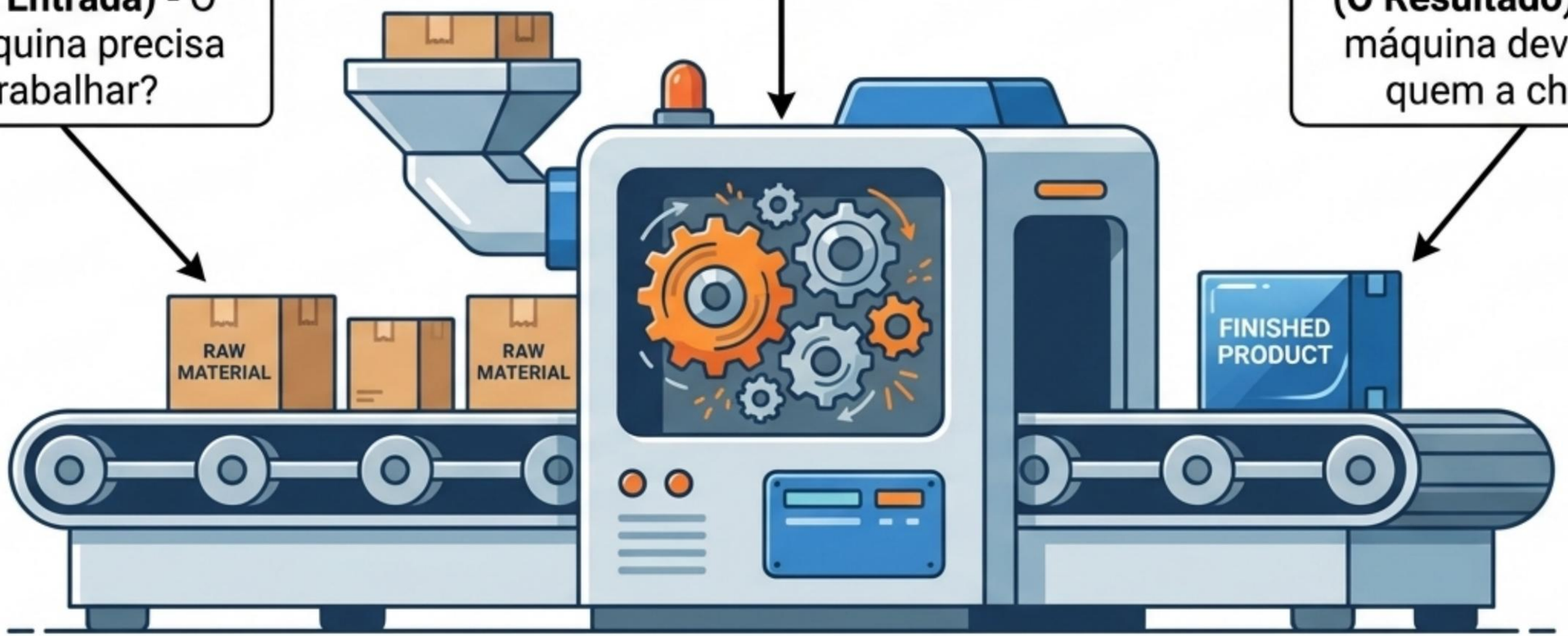
(Dados de Entrada) - O que a máquina precisa para trabalhar?

Corpo do Método

(Onde a mágica acontece) - A sequência de comandos e lógica.

Retorno

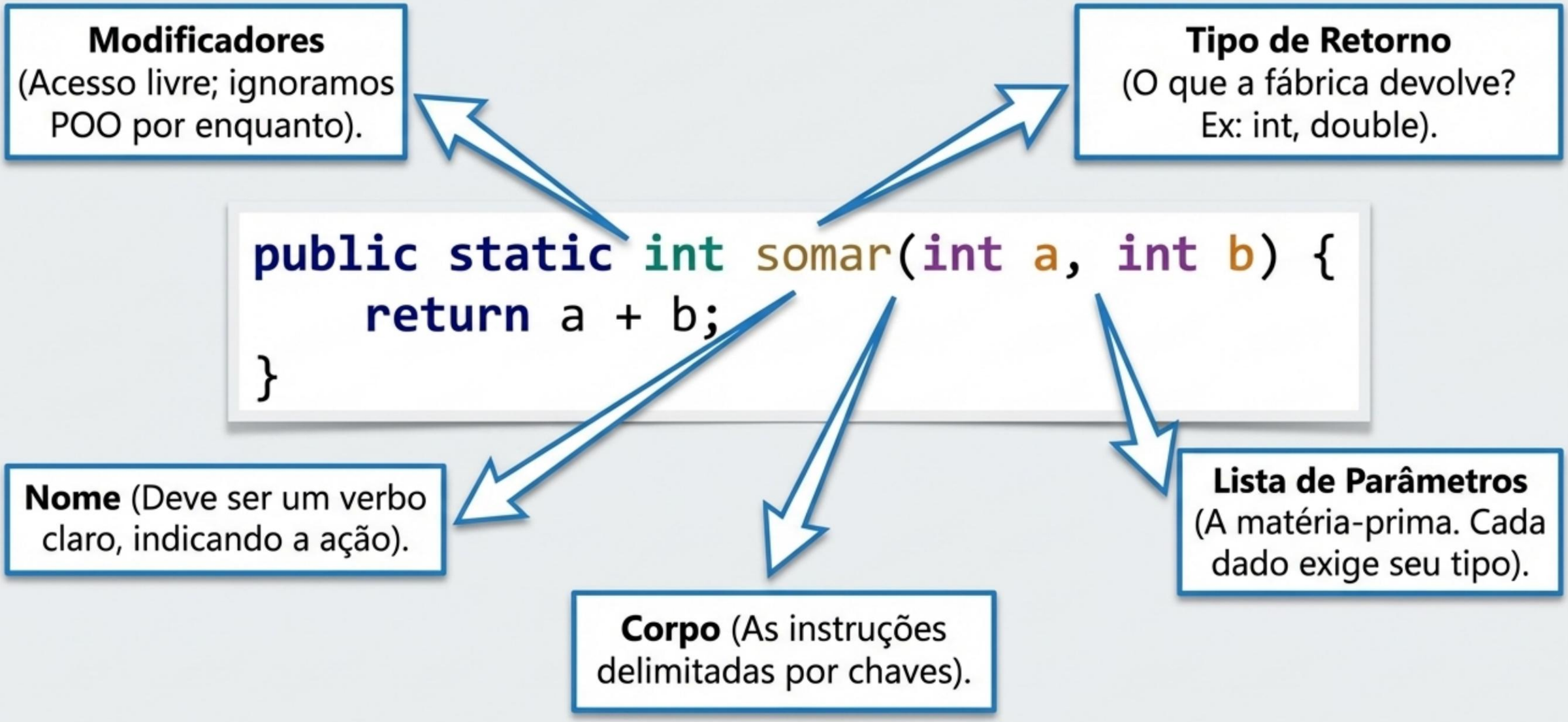
(O Resultado) - O que a máquina devolve para quem a chamou?



ronaldo.pb@ifpi.edu.br | (IFPI, 2026, Plano de Ensino)

NotebookLM

A Anatomia de uma Função (Método) em Java



NotebookLM

Dissecando a Saída: Devolver vs. Executar

O Retorno **OBRIGATÓRIO**



Funções que **DEVOLVEM** um valor (Tipos primitivos: **int, double, boolean, String**).

```java
public static boolean isAprovado(double nota) {
    return nota >= 7.0; // A palavra 'return' é OBRIGATÓRIA
}
```

A Ação **VAZIA**



Funções que apenas **FAZEM** algo, mas não devolvem dados. A palavra mágica é **void** (vazio).

```java
public static void darBoasVindas(String nome) {
    System.out.println("Olá, " + nome); // Não há 'return'
}
```


NotebookLM

A Lista de Parâmetros: Especificando os Ingredientes

Zero Ingredientes

( )

```
public static void
exibirMenu()
```

A função não precisa de nenhuma informação externa para funcionar.

Um Ingrediente

(int x)

```
public static int
calcularQuadrado(int x)
```

Exige exatamente um dado. O tipo (ex: int) deve ser declarado explicitamente.

Múltiplos Ingredientes

(String nome, int idade)

```
public static void cadastrar
(String nome, int idade)
```

Separados por vírgula. Cada parâmetro nasce e morre de forma independente.

NotebookLM

O Ciclo de Vida: A Execução no Método Principal

Método main

```
1  public static void main(String[] args) {
2      // Execução começa...
3      int x = 10;
4      int y = 5;
5      int resultado = somar(x, y);
6      System.out.println(resultado);
7  }
```

O programa pausa e envia os dados

Função somar

Recebe a = 10 e b = 5.
Calcula 15.

Devolve o resultado

Método main Continua

A variável 'resultado' agora guarda 15. A execução segue.

```
System.out.println(resultado); // Imprime 15
```


NotebookLM

Passo a Passo: Construindo uma Calculadora de Médias

Passo 1: Criando a Função (Fora do main)

```
public static double calcularMedia(double n1, double n2) {
    double soma = n1 + n2;
    return soma / 2;
}
```

Note que definimos **double** como retorno, pois a divisão pode gerar números decimais.

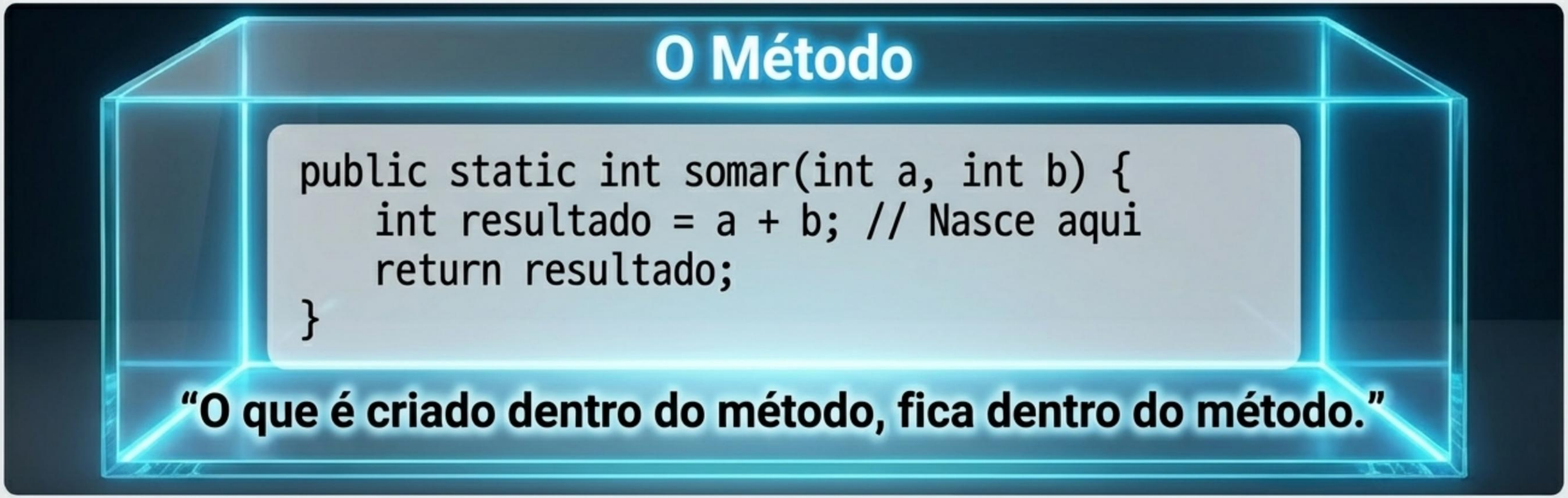
Passo 2: Chamando a Função (Dentro do main)

```
public static void main(String[] args) {
    double mediaFinal = calcularMedia(8.5, 7.0);
    System.out.println("A nota é: " + mediaFinal);
}
```

Os argumentos (8.5 e 7.0) são copiados para os parâmetros (n1 e n2) na exata ordem em que foram enviados.

NotebookLM

A Regra de Las Vegas: O Escopo Local das Variáveis




```
public static void main(String[] args) {
    System.out.println(resultado);
}
```

Erro de Compilação! O método `main` não enxerga variáveis locais de outras funções. Elas são destruídas assim que a função termina.

NotebookLM

O Mapa Mental: A Matriz de Tipos de Funções

Tem Retorno?	Tem Parâmetros?	
	Não	**Sim**
Void (Não)	**Ações Estáticas** `void exibirCabecalho()` Imprimir menus ou mensagens fixas.	**Ações Dinâmicas** `void imprimirTraco(int tamanho)` Executar tarefas visuais adaptáveis.
Retorna Tipo (Sim)	**Geradores** `int sortearNumero()` Criar ou capturar um dado novo do nada.	**Transformadores (A Clássica)** `boolean isPar(int numero)` Recebe dados, processa e devolve resultado.

ronaldo.pb@ifpi.edu.br | (Síntese Didática)

NotebookLM

Aplicação Prática: Dominando a Lógica Booleana

A Função Transformadora

```java
public static boolean isPar(int numero) {
    return (numero % 2 == 0);
}
```

Apenas uma linha de processamento elegante.

O Poder no main

```java
if (isPar(10)) {
    System.out.println("O número é par!");
}
```



A Mágica da Legibilidade! O código dentro do if lê-se quase como uma pergunta em português: 'Se é par (10), então...'. Isso esconde a complexidade matemática do resto do programa.

ronaldo.pb@ifpi.edu.br | (Aula 1.1, IFPI)

NotebookLM

Expandindo o Corpo: Laços de Repetição em Funções

```java
public static void gerarTabuada(int
    numero) {
    System.out.println("Tabuada do " +
    numero);

    for (int i = 1; i <= 10; i++) {
        System.out.println(numero + " x "
          + i + " = " + (numero * i));
    }

}
```

Liberdade Total:

- Uma função pode conter `if/else`, `while`, `for`, ou `switch`. Tudo o que você já aprendeu se aplica aqui.

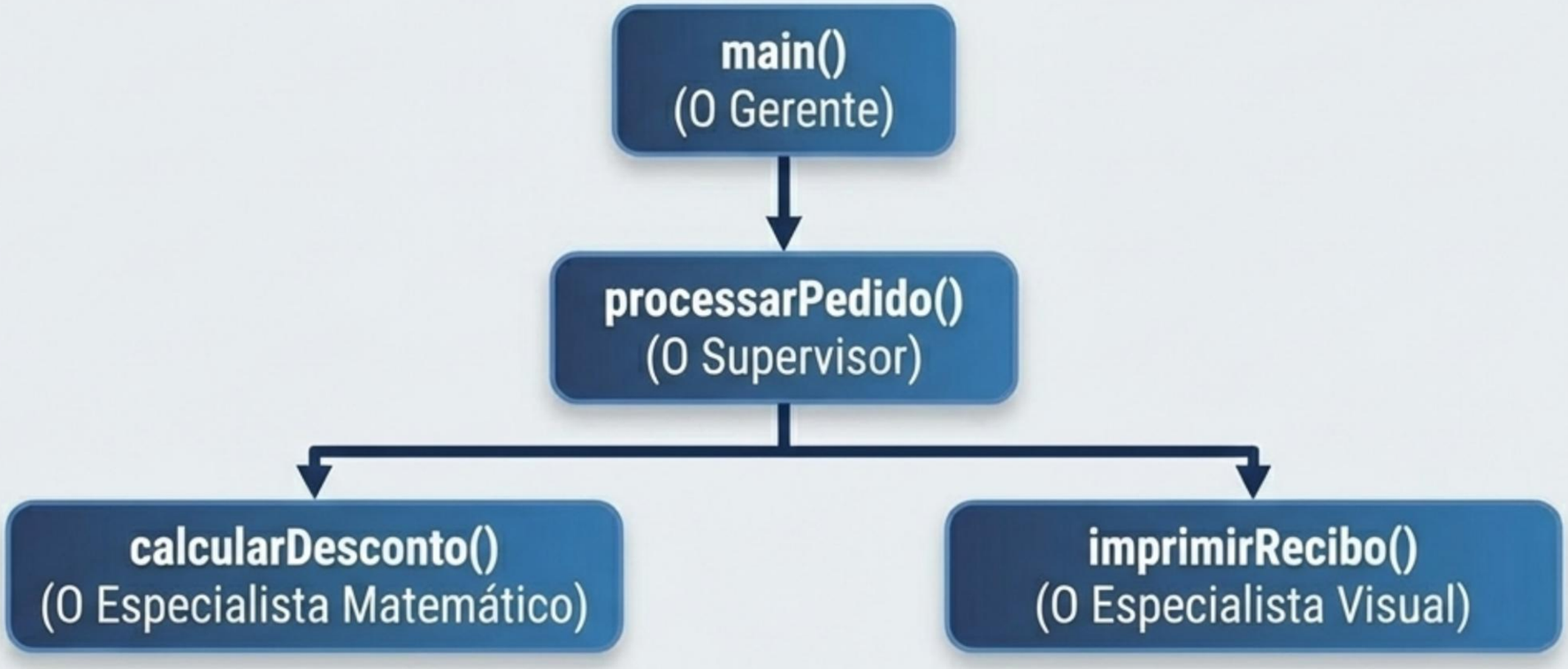
Reuso Extremo:

- Precisa da tabuada do 5 e do 9?
- Basta chamar `gerarTabuada(5);` e depois `gerarTabuada(9);` no `main`. Nenhuma linha de for repetida!

ronaldo.pb@ifpi.edu.br | (Lista 3, Exercício 2)

NotebookLM

A Arte da Delegação: Funções que Chamam Funções



A **verdadeira organização de código**. O método `main` age como um gerente de alto nível, delegando tarefas complexas para especialistas focados. O código torna-se um sumário executivo.

ronaldo.pb@ifpi.edu.br | (BEZERRA, 2015)

NotebookLM

As Regras de Ouro para Funções Impecáveis



Regra 1: Foco Único

Uma função, uma tarefa. Não crie uma função que calcula o salário, imprime o recibo e envia um email ao mesmo tempo. Divida para conquistar.



Regra 2: Verbos Claros

Nomes são ações. Use o formato camelCase começando com verbos. Ruim: calculo(). Bom: calcularImposto() ou verificarStatus().



Regra 3: O Contrato do Tipo

Cumpra a promessa. Se você declarou que a função retorna um int, o Java exigirá um return com um número inteiro. Você não pode retornar uma String nem sair sem devolver nada.

ronaldo.pb@ifpi.edu.br | (Boas Práticas de Engenharia de Software)

NotebookLM

Referências Bibliográficas

- BEZERRA, Eduardo. Princípios de análise e projeto de sistemas com UML. 3. ed. Rio de Janeiro: Elsevier, 2015.
- DEITEL, Paul; DEITEL, Harvey. Java: como programar. 10. ed. São Paulo: Pearson Education do Brasil, 2016.
- IFPI. Plano de Ensino: Programação Orientada a Objetos. Campus Floriano, 2026.
- SCHILDT, Herbert. Java para iniciantes: crie, compile e execute programas Java rapidamente. 6. ed. Porto Alegre: Bookman, 2015.

NotebookLM